



Guía del kit de importación con workloader

Cómo pasar de un export de Explorer a workloads no gestionados y listas de IP cargados en el PCE, con reconciliación por IP y trazabilidad de cada corrida

Eric Roscher – Illumio LATAM Pre-Sales Engineering

Septiembre 2026

Guía de trabajo preparada por Eric Roscher – Illumio LATAM Pre-Sales Engineering — no es una publicación oficial de Illumio, Inc.

■ Tabla de contenidos

01	Qué es el kit y cómo encaja en el trabajo	3
02	Requisitos y una carpeta por PCE	4
03	Objetos de política y nomenclatura	5
04	La TUI paso a paso	7
05	El kit y workloader	9
06	Del CSV a los objetos del PCE	11
07	Solución de problemas y lista de control	12

■ Qué es el kit y cómo encaja en el trabajo

El kit de importación cierra el último tramo de un análisis de flujos: toma los workloads no gestionados y las listas de IP que propone el informe (ya en formato CSV) y los carga en el PCE con `workloader`, la herramienta de línea de comandos de código abierto que Illumio mantiene en GitHub¹. Lo que agrega el kit sobre `workloader` es lo que un import masivo no hace por sí solo: validar el CSV, reconciliar cada IP contra el inventario real del PCE, pedir confirmación antes de cada escritura, importar por lotes y dejar un registro completo de la corrida.

FLUJO COMPLETO: DEL EXPORT DE EXPLORER A LOS OBJETOS EN EL PCE



1. Export de la vista Explorer/Traffic con la ventana y los filtros de la prueba; se abre como texto, nunca en Excel sin importar las IP como texto.
2. Normalización (IP mutiladas, fechas, truncamiento), criterio workload no gestionado frente a lista de IP, evidencia por puerto, proceso y volumen, modelo de etiquetas y hallazgos.
3. El informe entrega los CSV en el contrato de `wkld-import` e `ipl-import`, con la nomenclatura `R_/A_/E_/L_` y el comentario de evidencia en `description`.
4. El kit llama a los subcomandos de `workloader`: primero solo lectura (inventario), después dry run y, tras confirmar, escritura por lotes.
5. `workloader` habla con la API REST del PCE usando el `pce.yaml` de la carpeta de trabajo.

UNA CARPETA POR PCE

`workloader` lee `./pce.yaml` de la carpeta donde se ejecuta (salvo que se indique `--config-file` o `ILLUMIO_CONFIG`)², y el kit escribe ahí mismo `runs/<timestamp>/`. Si dos clientes comparten carpeta, el error posible es importar los objetos de uno en el PCE del otro. Usá una carpeta por PCE, con su propio `pce.yaml`, sus CSV y sus corridas; la sección siguiente muestra la estructura.

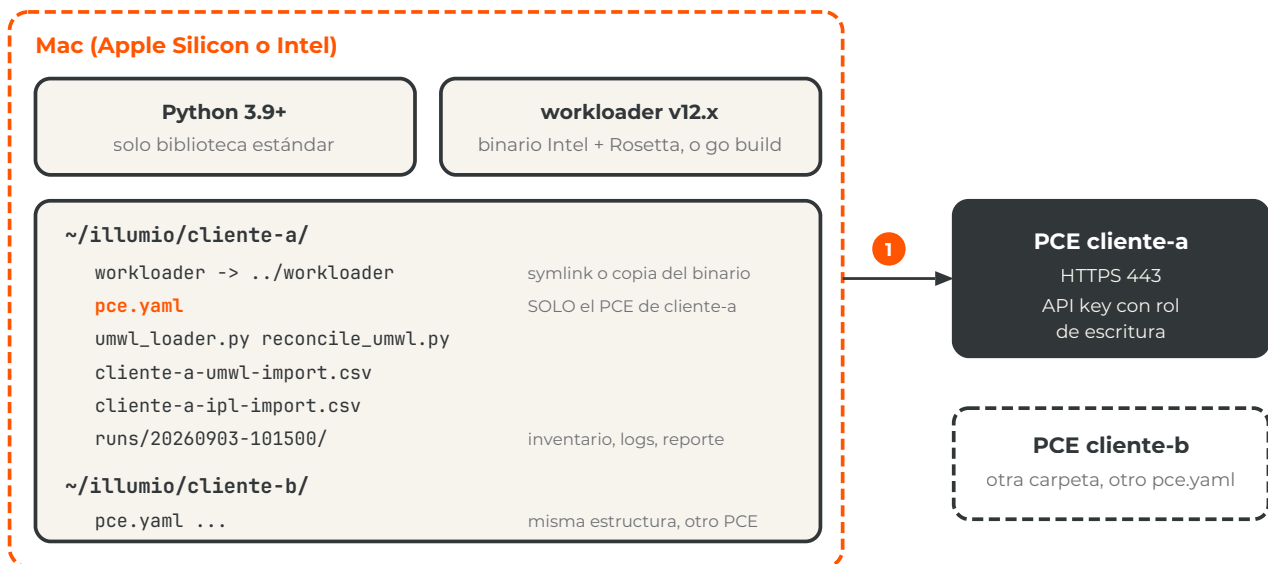
FUENTES — DOCUMENTACIÓN OFICIAL

1. `workloader` (brian1917) — README, releases v12.1.9 — github.com/brian1917/workloader
2. `workloader` — `cmd/root.go`: orden de búsqueda del archivo de configuración (`--config-file`, `ILLUMIO_CONFIG`, `./pce.yaml`) y flags persistentes — github.com/brian1917/workloader/blob/master/cmd/root.go

■ Requisitos y una carpeta por PCE

El kit no instala nada en el sistema: es un script de Python con la biblioteca estándar más el binario de workloader en la carpeta de trabajo. Lo que sí necesita es una identidad con permiso de escritura en el PCE y acceso HTTPS a su API.

REQUISITOS Y ESTRUCTURA DE CARPETAS: UNA POR PCE



1. workloader autentica con el `pce.yaml` de la carpeta actual; si el archivo tiene más de un PCE, pasar siempre `--pce <nombre>`. El kit muestra el resultado de `pce-list` antes de seguir.

COMPONENTE	DETALLE
macOS	El release de macOS (<code>mac-v12.1.9.zip</code>) es un binario Intel que corre con Rosetta ² ; para uno nativo, <code>brew install go</code> y <code>go build</code> desde el clon. La TUI ofrece ambas rutas.
Python 3.9+	Solo biblioteca estándar; no hay <code>pip install</code> .
workloader v12.x	Binario en <code>./workloader</code> , en el PATH o con <code>--workloader</code> . Al descargarlo, quitar la cuarentena: <code>xattr -d com.apple.quarantine workloader</code> .
API key del PCE	Menú de usuario → My API Keys → Add ; guardar <i>Authentication Username</i> (<code>api_...</code>) y <i>Secret</i> . Hereda el rol del usuario; para crear workloads, etiquetas y listas de IP hace falta Global Organization Owner o Global Administrator ^{2, 4} . En SaaS el <code>org id</code> es el número de la URL.
Red	HTTPS desde la Mac al FQDN del PCE (443 en SaaS). workloader no necesita nada más ³ .

PCE.YAML CONTIENE LA API KEY

No lo copies entre carpetas ni lo subas a un repositorio. El `.gitignore` del kit lo excluye junto con el binario, los logs y `runs/`.

FUENTES — DOCUMENTACIÓN OFICIAL

1. `workloader v12.1.9` — assets del release (mac-v12.1.9.zip, linux_amd64, linux_arm, windows) — github.com/brian1917/workloader/releases/tag/v12.1.9
2. Illumio Core 24.2 — REST API — API Keys (My API Keys, la key hereda el rol del usuario) — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/REST-APIs/out/en/rest-apis/authentication-and-api-user-permissions/api-keys.html
3. `workloader` — README: "The only requirements for workloader are the ability to run the executable and connect to the PCE over HTTPS" — github.com/brian1917/workloader
4. Illumio Core 25.4 — Admin — Roles, Scopes, and Granted Access — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Admin/out/en/pce-administration/role-based-access-control/roles,-scopes,-and-granted-access.html

■ Objetos de política y nomenclatura

El kit crea dos tipos de objeto y, como efecto secundario, valores de etiqueta. Saber qué es cada uno explica las decisiones del análisis y las de la TUI.

OBJETO	QUÉ ES Y CUÁNDO SE USA
Workload no gestionado (UMWL)	Entidad de red sin VEN que se da de alta en el PCE para escribir reglas sobre ella; la política se aplica con las reglas del lado que tiene VEN ¹ . Para servidores concretos con rol identificable (DNS, NTP, Zabbix, Splunk, NetBackup, bases de datos, balanceadores, bastiones, escáneres). Uno por IP, con etiquetas.
Lista de IP	Colección estática de direcciones, rangos y FQDN ² . Para peers amplios o que no son servidores : VLAN de usuarios, subredes completas, metadata de nube (169.254.169.254), NTP público, consolas SaaS. También como atajo para escribir hoy una regla que migra a etiquetas después.
Etiquetas y tipos	Cuatro tipos por defecto (Role, Application, Environment, Location) más los que se creen en el PCE ³ . workloader crea valores nuevos, no tipos : una columna que no sea un tipo existente se ignora en silencio (la TUI lo detecta).
Servicios	Objetos de puerto y protocolo. El informe los propone para las reglas; el kit no los crea (se cargan con <code>svc-import</code> o desde la consola).

Convención de nombres

CAMPO	REGLA	EJEMPLO
Etiquetas	Prefijo por tipo: R_ rol, A_ aplicación, E_ entorno, L_ ubicación	R_DNS · A_CoreInfra · E_Prod · L_OCI
name	Rol descriptivo seguido de la IP; es lo que se ve en la consola y lo que hace única a la fila	Zabbix Server 10.43.43.21
hostname	Vacío. Los FQDN de los exports pueden estar anonimizados; un hostname inventado confunde	(en blanco)
interfaces	eth0:<ip> ; varias interfaces separadas por ;	eth0:10.43.43.21
description	Prefijo [<grupo> P<prioridad> conf:<Alta Media Baja>] y el comentario con la evidencia	[C3 P1 conf:Alta] Servidor Zabbix: consulta 10050/TCP...

POR QUÉ LA PRIORIDAD VA EN LA DESCRIPCIÓN

Es el único campo libre que viaja con el objeto al PCE y que se puede leer después en la consola. El kit la usa para filtrar (`--priority 1`) sin necesitar columnas extra que workloader ignoraría.

FUENTES — DOCUMENTACIÓN OFICIAL

1. Illumio Core 24.5 — Security Policy — Workload Setup Using PCE Web Console (Add Unmanaged Workload) — product-docs-repo.illumio.com/Tech-Docs/Core/24.5/Security-Policy/out/en/security-policy-guide-24-5/workloads/workload-setup-using-pce-web-console.html
2. Illumio Core 24.2 — Getting Started — Policy Objects — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Getting-Started/out/en/policy-overview/policy-objects.html
3. Illumio Core 25.4 — Security Policy — Create a Label Type — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Security-Policy/out/en/security-policy-objects/about-labels-and-label-groups/label-types/create-a-label-type.html

■ La TUI paso a paso

Dos comandos cubren el uso normal. El primero se corre una vez por carpeta; el segundo, una vez por CSV.

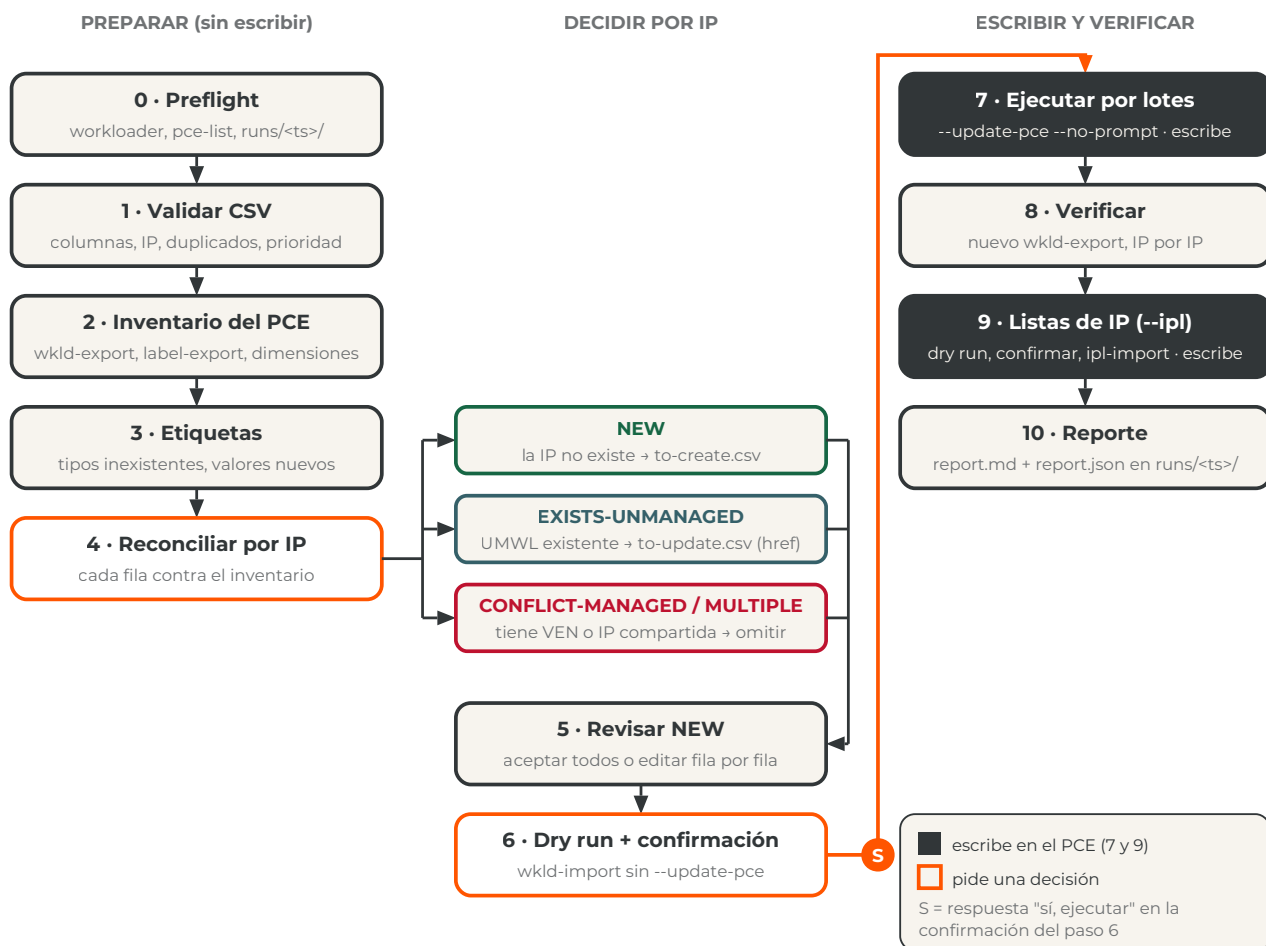
TERMINAL · CARPETA DEL CLIENTE

```
# 1. instalar/verificar workloader y configurar pce.yaml (una vez por PCE)
python3 umwl_loader.py --setup-only

# 2. cargar los workloads de prioridad 1 y, al final, las listas de IP
python3 umwl_loader.py cliente-a-umwl-import.csv --ipl cliente-a-ipl-import.csv --priority 1

# otros flags: --pce NOMBRE --workloader RUTA --chunk 20 --runs ./runs
```

SECUENCIA DE LA TUI Y PUNTOS DE DECISIÓN



Qué decide cada paso

PASO	DECISIÓN O RESULTADO
3 · Etiquetas	Columna que no es tipo de etiqueta del PCE: descartarla o salir y crear el tipo con <code>label-dimension-import</code> . Valores nuevos: se listan y se confirman antes de seguir.
4 · Reconciliar	Para cada EXISTS o CONFLICT: actualizar etiquetas y descripción del existente (por <code>href</code>), omitir , crear igual (duplicado consciente), renombrar y actualizar , o aplicar la misma decisión a todos los del mismo tipo. Por defecto: actualizar los EXISTS-UNMANAGED, omitir los CONFLICT.
5 · Revisar NEW	Aceptar todos o revisar uno por uno: conservar, editar (name, hostname, description, etiquetas), omitir, aceptar el resto.
6 · Dry run	Se muestran las líneas del log de workloader con lo que crearía o cambiaría. La pregunta "¿Ejecutar contra el PCE?" tiene como respuesta por defecto no : hay que escribir "s" para escribir.
7 · Lotes	Cada lote de <code>--chunk</code> filas es una llamada a <code>wkld-import</code> . Si un lote falla: reintentar, saltarlo (queda registrado) o abortar. El CSV del lote queda en <code>runs/<ts>/create-chunkNNN.csv</code> .

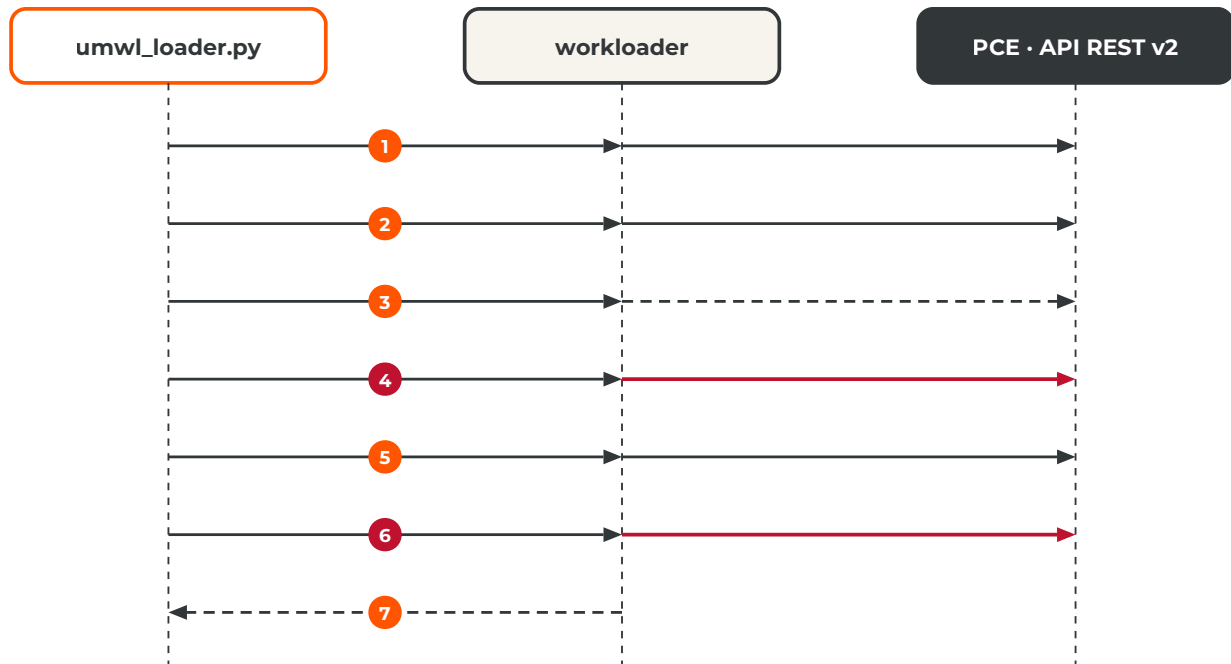
FUENTES — DOCUMENTACIÓN OFICIAL

1. workloader — README: `--update-pce` y `--no-prompt` ("auto-accept this prompt, as would be needed in automation"); logs en `workloader.log` — github.com/brian1917/workloader
2. workloader — `cmd/root.go`: flags persistentes `--pce`, `--log-file`, `--update-pce`, `--no-prompt`, `--config-file` — github.com/brian1917/workloader/blob/master/cmd/root.go

■ El kit y workloader

El kit no habla con la API del PCE: orquesta subcomandos de workloader y lee sus logs. Cada llamada se ejecuta con `--log-file runs/<ts>/<paso>.log` y, si se indicó, con `--pce <nombre>`. Sin `--update-pce`, ningún comando de workloader modifica el PCE¹; el kit solo agrega ese flag (junto con `--no-prompt`) en los pasos 7 y 9, después de la confirmación explícita.

QUIÉN LLAMA A QUIÉN: KIT, WORKLOADER Y PCE



Trazo rojo = escritura en el PCE (solo tras confirmar). Trazo punteado = solo lectura / log.

1. `pce-list` (y `pce-add --api-key ...` si no hay `pce.yaml`); workloader autentica con la API key y el kit prueba la conexión con un `label-dimension-export`.
2. `wkld-export`, `label-export`, `label-dimension-export` a `runs/<ts>/`: inventario de workloads (con `managed`, `ven_href`, `interfaces`, `public_ip`), etiquetas y tipos. Solo lectura.
3. `wkld-import to-create.csv --umwl --update=false` y `wkld-import to-update.csv` sin `--update-pce`: workloader lee el PCE para calcular el plan y lo escribe en el log; no modifica nada.
4. Por cada lote: `wkld-import create-chunkNNN.csv --umwl --update=false --update-pce --no-prompt` (y lo mismo para los de actualización, que hacen match por `href`). Crea workloads no gestionados y los valores de etiqueta que falten.
5. `wkld-export` de nuevo: el kit comprueba que cada IP creada aparece en el inventario.
6. Con `--ipl`: `ipl-import archivo.csv` (dry run) y, tras confirmar, `ipl-import archivo.csv --update-pce --no-prompt`.
7. Cada llamada deja su log en `runs/<ts>/`; el kit filtra las líneas "to be created", "bulk create workload successful", "[WARN]", "[ERROR]" y las muestra en pantalla.

Los mismos pasos sin la TUI

TERMINAL · EQUIVALENTES MANUALES (MISMA CARPETA, MISMO PCE.YAML)

```
# PCE (una vez por carpeta) e inventario
./workloader pce-add --api-key --name cliente-a --fqdn pce.cliente-a.example --port 443 \
  --api-user api_xxx --api-secret '...' --org 1
./workloader wkld-export --output-file pce-workloads.csv
python3 reconcile_umwl.py pce-workloads.csv cliente-a-umwl-import.csv

# dry run (leer workloader.log), después escribir
./workloader wkld-import cliente-a-umwl-import-to-create.csv --umwl
./workloader wkld-import cliente-a-umwl-import-to-create.csv --umwl --update-pce

# existentes: solo etiquetas y descripción, match por href
./workloader wkld-import cliente-a-umwl-import-existing.csv --update-pce
./workloader ipl-import cliente-a-ipl-import.csv --update-pce
```

PCE-ADD NECESITA --API-KEY PARA USAR UNA API KEY

En el código de workloader, `--api-user`, `--api-secret` y `--org` solo se leen cuando está presente el flag `--api-key`; sin él, el comando pide correo y contraseña². La versión del kit publicada en el repositorio ya lo incluye.

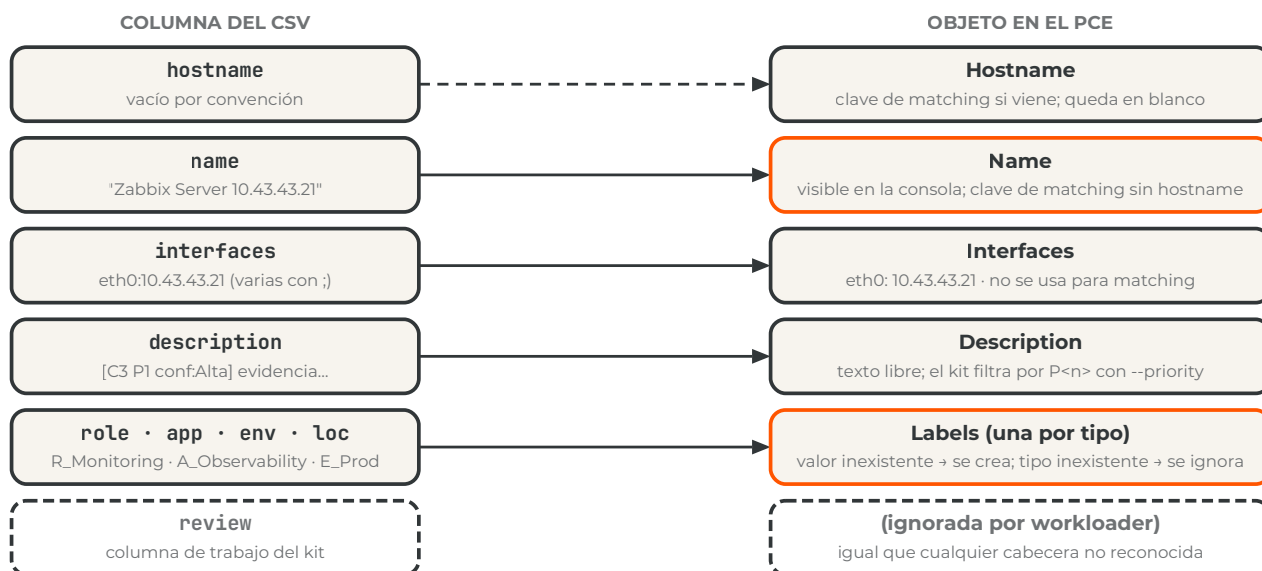
FUENTES — DOCUMENTACIÓN OFICIAL

1. workloader — README: "When a command modifies resources in the PCE, workloader does not trigger the action unless the `--update-pce` flag is included" — github.com/brian1917/workloader
2. workloader — cmd/pcemgmt/addpce.go: flags de pce-add (`--api-key`, `--api-user`, `--api-secret`, `--org`, `--disable-tls-verification`) — github.com/brian1917/workloader/blob/master/cmd/pcemgmt/addpce.go
3. workloader — cmd/wkldexport/headers.go: columnas del export (hostname, name, interfaces, public_ip, href, managed, ven_href...) — github.com/brian1917/workloader/blob/master/cmd/wkldexport/headers.go

■ Del CSV a los objetos del PCE

`wkld-import` reconoce un conjunto fijo de cabeceras (`href`, `hostname`, `name`, `interfaces`, `public_ip`, `description`, `enforcement`, `visibility` y otras) más **una columna por tipo de etiqueta existente**; el resto se ignora¹. El diagrama muestra a qué campo del workload va cada columna.

COLUMNAS DEL CSV Y CAMPOS DEL WORKLOAD NO GESTIONADO



Matching: por href, hostname o name, nunca por IP

Antes de crear, workloader busca si la fila ya existe: por `href` si la columna viene, si no por `hostname`, si no por `name` (`--match` fuerza uno de ellos)¹. La IP no participa. Por eso dos filas con el mismo `name` y sin `hostname` son el mismo workload para workloader (el kit les agrega la IP como sufijo), y por eso un UMWL que ya existe con otro nombre se crearía duplicado si no se reconcilia antes por IP contra el `wkld-export`: ese es el paso 4 de la TUI y el único trabajo de `reconcile_umwl.py`.

Listas de IP. El contrato de `ipl-import` es `name,description,include,exclude,fqdns`: `name` es la clave de matching cuando no hay `href` (si existe, actualiza; si no, crea)² y por convención va en minúsculas con prefijo `ipl-`; `include` y `exclude` llevan IP, CIDR o rangos separados por `;`; `fqdns`, nombres cuando corresponde; `description`, el razonamiento del informe.

FUENTES — DOCUMENTACIÓN OFICIAL

- workloader — `cmd/wkldimport/cmd.go`: cabeceras reconocidas, formato de interfaces, `--umwl`, `--update`, `--match` (`href`, `hostname`, `name`, `external_data`) — github.com/brian1917/workloader/blob/master/cmd/wkldimport/cmd.go
- workloader — `cmd/iplimport/iplimport.go`: cabeceras `name`, `description`, `include`, `exclude`, `fqdns`; separador `;;`; matching por `href` y `name` — github.com/brian1917/workloader/blob/master/cmd/iplimport/iplimport.go

■ Solución de problemas y lista de control

SÍNTOMA	CAUSA Y SOLUCIÓN
macOS no abre <code>workloader</code> ("no se pudo verificar el desarrollador")	Cuarentena de Gatekeeper al descargar el zip. <code>xattr -d com.apple.quarantine ./workloader && chmod +x ./workloader</code> . La TUI lo hace al descargar.
Apple Silicon	El release es Intel y corre con Rosetta 2. Para un binario nativo: <code>brew install go</code> , clonar el repositorio y <code>go build -o ../workloader .</code> ; la TUI ofrece la opción.
<code>pce-add</code> pide correo y contraseña	Falta <code>--api-key</code> ; sin ese flag se ignoran <code>--api-user</code> , <code>--api-secret</code> y <code>--org</code> .
El dry run funciona y el import devuelve 401/403	La API key hereda el rol del usuario. Hace falta un rol global con escritura (Global Organization Owner o Global Administrator) o una service account equivalente. Revisar también el <code>org id</code> .
<code>pce-list</code> muestra otro cliente	Carpeta equivocada o <code>ILLUMIO_CONFIG</code> apuntando a otro archivo. Una carpeta por PCE; <code>unset ILLUMIO_CONFIG; --pce <nombre></code> .
Se cargó un solo workload de varios con el mismo nombre	Mismo <code>hostname</code> (o mismo <code>name</code> sin hostname) = mismo workload para <code>workloader</code> . Hacer único el <code>name</code> ; la TUI agrega la IP como sufijo.
Una columna de etiqueta no se aplicó	El tipo no existe en el PCE. Crearlo en la consola o con <code>./workloader label-dimension-import tipos.csv --update-pce</code> (CSV con <code>key</code> y <code>display_name</code>) ¹ y volver a correr.
El export de Explorer tiene exactamente N filas redondas	Está truncado al límite de la consulta. Subir el límite, filtrar por aplicación, excluir escáneres, acortar la ventana y volver a exportar; la consola muestra hasta 100.000 resultados y el CSV hasta 200.000 en un PCE standalone ² .
Error TLS con PCE on-prem	Certificado interno. Instalar la CA en el llavero; <code>--disable-tls-verification true</code> solo en laboratorio.

Antes de escribir en el PCE

- ☐ **Carpeta correcta** — `pce-list` muestra el PCE del cliente y no hay `ILLUMIO_CONFIG` definida.
- ☐ **CSV en el formato v2** — hostname vacío, `name` único con IP, `interfaces` como `eth0:<ip>`, etiquetas con prefijo, prioridad en la descripción.
- ☐ **Tipos de etiqueta** — cada columna de etiqueta del CSV existe en el PCE (paso 3 sin advertencias).

- ☐ **Reconciliación revisada** — ningún CONFLICT-MANAGED se actualiza sin haberlo mirado; los EXISTS-UNMANAGED conservan el nombre del PCE salvo decisión explícita.
- ☐ **Dry run leído** — las líneas "to be created" coinciden con lo esperado y no hay "[ERROR]".
- ☐ **Lote pequeño la primera vez** — `--chunk 5` en la primera corrida de un PCE nuevo para acotar el impacto de un error.
- ☐ **Reporte archivado** — `runs/<ts>/report.md` queda en la carpeta del cliente; no se sube al repositorio.

FUENTES — DOCUMENTACIÓN OFICIAL

1. [workloader — cmd/labeldimension/import.go: label-dimension-import \(cabeceras key, display_name, href...\) — github.com/brian1917/workloader/blob/master/cmd/labeldimension/import.go](https://github.com/brian1917/workloader/blob/master/cmd/labeldimension/import.go)
2. [Illumio Core 25.4 — Visualization — About the Visualization Tools \(Explore, Traffic, límites de resultados, consultas asíncronas\) — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Visualization/out/en/visualization-tools/about-the-visualization-tools.html](https://product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Visualization/out/en/visualization-tools/about-the-visualization-tools.html)
3. [Illumio Core 22.5 — REST API — Explorer: traffic_flows/async_queries, max_results 200.000 — product-docs-repo.illumio.com/Tech-Docs/Core/22.5/REST-APIs/out/en/core-22-5-rest-api-developer-guide/visualization/explorer.html](https://product-docs-repo.illumio.com/Tech-Docs/Core/22.5/REST-APIs/out/en/core-22-5-rest-api-developer-guide/visualization/explorer.html)

Código, CSV de ejemplo y esta guía



Repositorio illumio-workloader-import-kit: README con el paso a paso de Explorer y los prompts para replicar el análisis.

ACERCA DE ESTE DOCUMENTO

Esta guía fue preparada por Eric Roscher – Illumio LATAM Pre-Sales Engineering para explicar el uso del kit de importación de workloads no gestionados y listas de IP con workloader. Es un material de apoyo para el destinatario y no constituye una publicación oficial de Illumio, Inc.; no modifica ni reemplaza la documentación oficial del producto, los acuerdos contractuales ni los alcances de trabajo (SOW). Valide todos los detalles técnicos contra la documentación oficial correspondiente a sus versiones específicas antes de actuar sobre ellos. Illumio y el logotipo de Illumio son marcas de Illumio, Inc., utilizadas aquí para identificar el tema tratado.

© 2026 Illumio, Inc. All Rights Reserved.